

文档编号：人大信息学院学生综合服务与党团管理系统 – SDS – <\*\*\*.\*\*\*>

# 人大信息学院学生综合服务与党团 管理系统 软件设计规格说明书

日期：2026.5.12

# 文档变更历史记录

| 序号 | 变更日期        | 变更人员 | 变更内容详情描述    | 版本    |
|----|-------------|------|-------------|-------|
| 1  | 2026. 5. 10 | 路嘉洁  | 填写第一部分      | 1. 0  |
| 2  | 2026. 5. 11 | 范逍   | 填写第二部分      | 1. 01 |
| 3  | 2026. 5. 12 | 陈娅润  | 填写第三部分，修改格式 | 1. 02 |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |
|    |             |      |             |       |

# 目录

|                      |    |
|----------------------|----|
| 1、引言 .....           | 4  |
| 1.1 编写目的 .....       | 4  |
| 1.2 读者对象 .....       | 4  |
| 1.3 软件项目概述 .....     | 6  |
| 1.4 文档概述 .....       | 7  |
| 1.5 定义 .....         | 9  |
| 1.6 参考资料 .....       | 12 |
| 2、软件设计约束 .....       | 13 |
| 2.1 软件设计目标和原则 .....  | 13 |
| 2.2 软件设计的约束和限制 ..... | 13 |
| 3、软件设计 .....         | 15 |
| 3.1 软件体系结构设计 .....   | 20 |
| 3.2 用户界面设计 .....     | 24 |
| 3.3 用例设计 .....       | 30 |
| 3.4 类设计 .....        | 35 |
| 3.5 数据设计 .....       | 42 |
| 3.6 部署设计 .....       | 46 |

# 1、引言

## 1.1 编写目的

本文档为“中国人民大学信息学院学生综合服务与党团管理平台”的软件设计规格说明书(SDS)。编写本文档旨在将《软件需求规格说明书(SRS)》所确立的业务逻辑与性能指标，转化为标准化的技术实现路径，为后续开发与运维提供技术纲领。

具体目标涵盖以下五个维度：

**1. 确立技术蓝图:**针对 SRS 中定义的智能问答、党团流程、电子证明、精准推送及学业预警五大功能模块，提供从业务逻辑到物理实现的精确映射。通过规范体系结构与接口标准，确保系统在多端协同开发中具备排他性的技术准则。

**2. 指导技术适配:**针对中国人民大学信息学院特定的信息化生态，详细规定系统如何深度适配人大金仓(Kingbase)国产数据库及学院 Linux 服务器环境。通过底层架构的兼容性设计，确保系统在国产软硬件环境下的长期运行稳定性。

**3. 规范权限体系:**将分级审批与数据脱敏<sup>[1]</sup>等管理需求转化为具体的 RBAC 逻辑模型<sup>[2]</sup>。通过对加密算法及鉴权逻辑的详细规约，确保学生身份证号、处分记录等敏感数据在系统全生命周期的流动符合学校安全管理规定。

**4. 强化质量管理:**为开发团队提供统一的编码规范与设计类定义，明确严禁“补丁式”开发的技术底线。通过对异常容错与审计日志的设计，确保系统代码具备高可读性、可维护性及可审计性。

**5. 明确验收准则:**本文档定义的体系结构及数据库模型将作为软件工程课

程项目评审、答辩及甲方(学院分团委)最终验收的正式依据。通过对技术实现深度的量化描述，确保交付产物与 SRS 预设的需求指标高度对齐。

## 1.2 读者对象

本文档面向“中国人民大学信息学院学生综合服务与党团管理平台”开发、设计、评审及管理全过程中的各类核心参与人员，具体阅读对象及使用场景如下：

**1. 系统研发人员：**包含前端(微信小程序)、后端(Spring Boot/FastAPI)及 AI 算法模块的全体开发成员。本文档为其提供系统架构逻辑、类设计规约及数据库物理模型，作为其进行底层代码实现、接口联调及模块集成的排他性技术依据。

**2. 质量保障与测试人员：**作为编写集成测试用例与安全性审计计划的核心准则，测试人员需依据本文档定义的四级权限校验逻辑、敏感数据脱敏规则及金仓数据库事务处理机制，验证系统实现的闭环性与运行稳定性。

**3. 课程评审与评估团队：**包含信息学院软件工程课程的指导教师、助教及最终答辩评审专家。本文档为其评估项目技术深度、架构合理性及对国产化数据库适配程度提供量化的审计依据。

**4. 业务需求审定方(甲方)：**包含学院分团委老师、辅导员及相关学生工作负责团队。其通过审阅本文档，核实系统设计方案是否精准映射了 SRS 中的管理诉求，尤其是入党流程追踪、证明自动生成及学业预警逻辑在技术层面是否具备严密性。

**5. 系统运维人员:**了解系统的部署架构、环境配置约束(如 Docker<sup>[3]</sup> 容器化方案)及金仓数据库的维护要求,确保平台在学院服务器环境下的稳定运行与后续技术升级。

## **1.3 软件项目概述**

**1.3.1 项目名称:** 人大信息学院学生综合服务与党团管理系统

**1.3.2 用户单位**

本项目用户单位为中国人民大学信息学院。系统服务对象涵盖全院约 1200 名在籍本科生与研究生,以及由学院分团委老师、班主任及相关学生工作负责人组成的管理团队。

**1.3.3 开发单位:** 萌萌软工少女团!

**1.3.4 软件项目需求描述**

本项目旨在构建一个集事务办理、信息服务与数据分析于一体的学院级“一站式”数字化服务窗口。其核心需求涵盖功能与性能两大维度:

### **1. 核心功能维度**

系统通过模块化解耦设计,深度覆盖以下五大业务领域:

**(1) 智能问答系统:**利用 AI 匹配技术构建学院政策知识库,实现奖助学金、学籍变动等高频政策的即时、精准回复。

**(2) 党团流程管理:**实现从入党申请人到正式党员全生命周期的可视化追踪与时间节点自动提醒。

(3) **信息精准推送**: 基于学生画像标签, 实现学院通知与就业/竞赛信息的分类聚合及多渠道精准触达。

(4) **电子审批流转**: 支持在线生成标准化证明文件及学院内部事务的多级在线审批, 具备完整的审计日志记录。

(5) **学业分析预警**: 支持 PDF 成绩单自动解析与培养方案比对, 为学生提供学分缺口提示与选课建议。

## 2. 关键性能与技术指标

(1) **环境适配性**: 系统必须全面适配人大金仓国产数据库及学院 Linux 服务器环境, 确保底层语法与事务处理的兼容性。

(2) **高并发鲁棒性**: 需支撑高峰期约 200 名师生同时在线请求, 普通业务查询响应时间需控制在 1.5 秒以内。

(3) **四级权限安全**: 严格执行基于角色 (RBAC) 的四级访问控制体系。涉及身份证号、处分记录等敏感数据需实施底层加密存储与前端动态脱敏展示, 确保数据安全性符合学校管理规范。

(4) **多端协同一致性**: 通过统一的后端 RESTful<sup>[4]</sup> 服务支撑, 确保微信小程序 (学生端) 与 PC 端 (管理员后台) 在数据交互中的强一致性。

## 1.4 文档概述

本文档作为“人大信息学院学生综合服务与党团管理系统”平台研发过程中的核心技术纲领, 旨在将《软件需求规格说明书 (SRS) 》中定义的抽象需求,

通过多维建模与逻辑规约，转化为可落地的物理架构与代码蓝图。共分为以下三个核心维度进行系统性阐述：

## 第一部分:引言

本章节确立了文档的编写背景与语境边界。通过对编写目的、读者对象的明确，确保研发、测试及评审方达成技术共识。此外，本部分详细定义了涉及国产数据库适配、安全性规约及工程指标等方面的关键术语，为全文的技术实施提供了标准化的词汇表。

## 第二部分:软件设计约束

本章节定义了系统研发必须遵循的“刚性准则”与“底线要求”，是确保项目符合人大信息学院管理规范的前提：

**目标与原则规约:**确立了“前后端分离”、“多端协同一致性”及“最小权限”等核心设计原则，明确禁止“补丁式”开发以保障系统的可持续维护性。

**环境与栈选型限制:**详细规定了系统对人大金仓国产数据库的深度适配要求，以及在 Linux 服务器环境下的容器化(Docker)运行约束。

**非功能性指标:**对系统在奖学金评定等业务高峰期的并发承载能力(约 200 人并行)、响应时延(1.5 秒以内)及安全性合规(四级权限体系)进行了量化定义。

## 第三部分:软件设计

本部分为本文档的技术精髓，通过六个子章节对系统进行了全方位的物理建模与逻辑拆解：

**体系结构设计(3.1):**规约了由接入层、应用服务层、公共支撑层及数据持

久化层组成的“四层架构模型”，确保了业务逻辑与底层数据的深度解耦。

**用户界面设计(3.2):**秉持“严谨、正式、简约”的人大审美风格，详细设计了小程序端与管理端的页面跳转逻辑图，并确立了“三次点击直达核心业务”的易用性准则。

**用例设计(3.3):**针对 SRS 中提出的 18 个核心功能点，选取了智能政策问答、分级审批流转、学业预警提示等关键业务，通过交互顺序图 (Sequence Diagram) 完整还原了从前端请求到后端处理再到数据反馈的动态闭环过程。

**类设计(3.4):**定义了包括 User、Student、PartyProgress、ApprovalRecord 在内的核心领域模型，明确了各实体间的关联关系及 Service 层的职责分配。

**数据设计(3.5):**展示了基于 Kinabase 语法的逻辑 E-R<sup>[6]</sup> 模型。特别设计了敏感信息加密存储方案及基于“追加写入”模式的审计日志表结构，以满足安全性与可追溯性要求。

**部署设计(3.6):**提供了基于 Nginx 与 API Gateway<sup>[6]</sup> 的统一入口部署方案，并说明了如何通过 Docker 容器化技术实现系统在学院内网环境下的快速分发与零配置运行。

## 1.5 定义

**[1]数据脱敏 (Data Masking):**指在不改变原始敏感数据存储的前提下，根据访问者权限，在界面展示层通过部分屏蔽(如“101..001X”)或动态隐藏技术，对身份证号、处分记录等敏感隐私信息进行模糊化处理，确保非授权角色无法获知原始完整数据。

[2]**RBAC 逻辑模型 (Role-Based Access Control)**:即基于角色的访问控制模型。通过将“权限”赋予特定“角色”，再将“角色”分配给“用户”，实现管理权限与具体身份的逻辑解耦。本项目利用该模型严密规约学院领导、管理老师、班团骨干、普通学生四级角色的访问边界。

[3]**Docker**:一种轻量级的开源容器化技术。本项目采用其进行软件环境的物理封装，确保系统包含代码、运行库及配置在内的全镜像能在学院 Linux 服务器上实现环境无关的跨平台迁移与零配置部署。

[4]**RESTful**:一种遵循表征状态转移 (Representational State Transfer) 风格的软件架构模式。系统移动端与 PC 端通过基于 HTTP 协议的无状态接口进行 JSON 格式的数据交互，旨在提高多端协同下的系统伸缩性与前后端解耦程度。

[5]**E-R 模型 (Entity-Relationship Model)**:实体-联系模型。作为业务需求向数据库实现的中间逻辑模型，其通过图形化方式精化描述系统底层人大金仓数据库中各类数据实体、属性及相互间的约束关系。

[6]**Nginx/API Gateway**:系统接入层的双重架构组件。Nginx 负责公网请求的负载均衡与反向代理;API Gateway 负责统一接口路由、服务鉴权及流量控制，作为系统后端服务的“安全门户”屏蔽底层服务细节。

[7]**HTTPS 协议**:基于 SSL/TLS 加密层的安全超文本传输协议。要求系统全链路通信必须通过非对称加密技术，确保学生隐私数据在不安全的公共网络环境传输过程中不被非法截获、篡改或重放。

[8]**WAR/Jar 包**:Java 语言编写的 Web 应用程序归档文件格式。本项目后端 Spring Boot 服务封装为 Jar 包,内嵌 Servlet 容器,作为系统核心业务逻辑的最终运行实体。

[9]**WXML/WXSS**:微信小程序专用框架中的标记语言与样式表语言。WXML 负责定义移动端页面结构,WXSS 负责规约页面视觉呈现(如人大的严谨配色与排版布局),二者共同构建学生入口的交互层。

[10]**Vue.js/React**:主流的前端 JavaScript 组件化框架。用于构建 PC 管理端单页面应用(SPA),支持管理员在无需刷新页面的情况下完成复杂的报表统计、批量审批及数据导入导出交互。

[11]**Spring Boot (Java)**:本项目后端业务逻辑的核心支撑框架。其采用约定优于配置的设计理念,支持构建具备高内聚、分层架构特征的微服务,承载系统复杂的审批流转与学业分析逻辑。

[12]**FastAPI (Python)**:高性能的异步 Web 开发框架。本项目用于集成基于 Python 开发的 AI 算法模块,通过其原生的异步 IO 特性支持政策问答引擎的大并发查询与毫秒级推理响应。

[13]**MySQL、PostgreSQL、Oracle**:主流商用或开源关系型数据库。本文档提及此类数据库,旨在作为本项目唯一指定存储引擎——人大金仓的对比参考,强调系统严禁使用任何与其不兼容的非标语法或闭源特性。

[14]**Tomcat/JVM**:Tomcat 是支持 Java Servlet 规范的应用服务器,JVM(Java 虚拟机)是 Java 字节码的运行环境。二者共同构成本项目后端业务逻辑

辑服务的底层运行时的基础架构。

[15]**Postman**: 标准化的接口测试与联调工具。用于模拟各类 HTTP 请求, 验证后端各业务模块返回的 JSON 数据结构是否严格对齐《API 接口文档》的技术规约。

[16]**死锁 (Deadlock)**: 指在多端并发(如高峰期评奖评优)操作金仓数据库时, 两个或多个事务因互相持有对方所需的资源锁而产生的永久僵持状态。本项目通过优化索引设计与事务原子性规约, 确保在高并发场景下规避此类故障。

## 1.6 参考资料

1. Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[J]. arXiv preprint arXiv:2312.10997, 2023, 2(1): 32.

来源: <https://arxiv.org/abs/2312.10997>

2. Martínez-Fernández S, Bogner J, Franch X, et al. Software engineering for AI-based systems: a survey[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 2022, 31(2): 1-59.

来源: <https://doi.org/10.1145/3487043>

3. Mohammad M. Resilient Microservices: A Systematic Review of Recovery Patterns, Strategies, and Evaluation Frameworks[J]. arXiv preprint arXiv:2512.16959, 2025.

来源: <https://arxiv.org/abs/2512.16959>

4. de Carvalho Junior M A, Bandiera-Paiva P. Health Information System Role - Based Access Control Current Security Trends and Challenges[J]. Journal of healthcare engineering, 2018, 2018(1): 6510249.

来源: <https://doi.org/10.1155/2018/6510249>

5. Munaye Y Y, Admass W, Belayneh Y, et al. ChatGPT in education: A systematic review on opportunities, challenges, and future directions[J]. Algorithms, 2025, 18(6): 352.

来源: <https://doi.org/10.3390/a18060352>

## 2、软件设计约束

### 2.1 软件设计目标和原则

#### 2.1.1 设计目标

本系统的设计目标是为人民大学信息学院构建一个覆盖学生日常事务全生命周期的“一站式”数字化服务平台，具体目标如下：

**实现用户需求，提升事务处理效率。** 系统须完整覆盖智能问答与政策知识库、党团事务流程管理、信息集成与精准推送、电子证明生成与审批流程、学业情况分析与预警五大核心业务模块，将原本依赖线下或人工处理的学院事务（如证明盖章、请假审批、入党进度追踪等）全面转化为线上电子化流程，切实解决当前流程不透明、信息滞后及重复性劳动多等痛点。

**构建四级权限安全体系，保障数据合规。** 系统须严格区分学院领导、管理老师、班团骨干与普通学生四类角色的访问权限，对身份证号、联系方式、处分记录等敏感数据实施加密存储与动态脱敏展示，确保系统设计符合学校及学院的数据安全管理规定。

**实现良好的可扩充性与可维护性。** 采用模块化架构，各功能单元（智能问答、党团管理、信息推送等）相互解耦、独立部署，以便后续快速迭代新功能（如出国申请、评价评优等），支持系统的长期演进与维护。

**适配国产化软硬件环境，确保稳定运行。** 系统须在人大金仓（Kingbase）

数据库及学院 Linux 服务器环境下稳定运行，前后端所有数据库操作须完整兼容国产数据库语法规则，不得依赖仅适用于其他商业数据库的专有特性。

**实现多端协同，保证数据一致性。** 微信小程序（移动端学生入口）与 PC 端（管理员后台）须在同一后端服务支撑下协同工作，通过事务管理机制确保多端并发操作下金仓数据库内的数据状态实时一致，杜绝数据错乱或流程死锁<sup>[16]</sup>。

### 2.1.2 设计原则

为实现上述设计目标，系统开发过程中须严格遵循以下一般性设计原则：

**前后端分离原则。** 系统必须采用前后端分离架构。移动端（微信小程序）、PC 管理端与服务端（Server）之间通过标准 RESTful 风格的 JSON 接口进行通信，各端职责明确，互不耦合，便于独立开发、测试与部署。

**模块化与高内聚低耦合原则。** 五大功能模块须作为独立的业务单元进行设计与开发，模块内部逻辑高度内聚，模块间通过标准接口交互，依赖关系尽量单向。严禁跨模块直接操作数据库或共享全局状态，以保证系统具备良好的可维护性与扩展性。

**最小权限原则。** 每类角色只能访问和操作其职责范围内的数据与功能。系统须在后端进行权限校验，不得仅依赖前端页面隐藏来限制越权访问。助教及三、四级角色严禁拥有审批权限，敏感信息须根据角色动态决定是否脱敏展示。

**安全优先原则。** 涉及学生隐私的所有敏感字段（身份证号、手机号、生

源地、处分记录等）须在数据库层加密存储，在传输层使用 HTTPS 协议<sup>[7]</sup>保护通信安全，在展示层按角色进行脱敏处理。智能问答模块的回复须优先基于官方知识库内容，严格规避大模型产生“幻觉”，不得将敏感信息直接暴露在 AI 回复中。

**可审计原则。** 所有管理员角色（老师、助教、班干）的关键操作（包括审批、数据修改、文件导入导出、敏感数据查阅等）须完整记录操作日志，日志须包含操作人、操作时间、操作内容及操作结果，确保审批操作和数据变更全程可追溯、可审计。

**容错与异常处理原则。** 系统在面对异常输入（如格式不符的 PDF 成绩单）、网络抖动或并发冲突时，须提供精准的错误定位反馈，而非直接崩溃或静默失败。审批流须支持 1 至 2 天内的撤销或重批机制，批量导入失败时须逐条给出错误定位提示。

**禁止“补丁式”开发原则。** 要求代码具备高可读性并配有完善的文档注释，严禁为修补局部逻辑漏洞而进行缺乏整体规划的临时性修改，所有变更须经过设计评审，确保系统的可持续维护性。

## 2.2 软件设计的约束和限制

### 2.2.1 运行环境要求

**硬件平台：**

**服务端：**系统须部署于学院提供的服务器，要求具备足够的计算与存储资源，以支撑高峰期约 200 名用户的并发访问；服务器与金仓数据库实例须

处于同一内网环境，保障数据库连接的低延迟与高可靠性。

**学生移动端：**须运行于支持微信客户端的主流智能手机（Android / iOS），对具体硬件配置不作严格限制，以兼顾全体学生用户。

**管理员 PC 端：**须运行于具备网络连接的个人电脑，对具体硬件配置不作严格限制。

### **操作系统：**

**服务端：**须兼容学院服务器所采用的 Linux 操作系统，部署产物（Docker 镜像或 WAR/Jar 包<sup>[8]</sup>）须在 Linux 环境下可直接运行，无需额外配置。

**移动端：**须兼容微信小程序所支持的 Android 及 iOS 系统版本。

**PC 管理端：**须兼容 Windows 及 macOS 操作系统下的主流桌面浏览器环境。

### **浏览器兼容性：**

PC 管理端须兼容 Chrome、Edge 及国产主流浏览器（如 360、QQ 浏览器极速模式等）。

移动端须在微信内置 WebView 及主流手机浏览器（Safari、Chrome）中流畅运行。

## **2.2.2 开发语言**

**移动端（微信小程序）：**采用微信小程序原生开发框架，使用 JavaScript/TypeScript 作为主要开发语言，配合 WXML 与 WXSS<sup>[9]</sup>进行页面结构与样式描述。

**PC 管理端：**采用 Vue.js 或 React<sup>[10]</sup>框架，使用 JavaScript/TypeScript

进行开发，构建处理复杂统计报表及文档导入导出的单页面应用（SPA）。

**服务端（后端）：** 采用 Spring Boot（Java）<sup>[11]</sup>或 FastAPI（Python）<sup>[12]</sup>

构建模块化后端服务；其中 Spring Boot 适用于复杂业务逻辑的分层处理，FastAPI 适用于高性能 API 接口响应及 AI 模块集成。

**AI 算法模块：** 采用 Python 进行开发，通过标准化 REST 接口与后端服务对接，负责政策问答引擎的知识库检索与结果过滤。

**数据库脚本：** 须使用兼容人大金仓（Kingbase）语法规则的 SQL 进行数据库结构定义及初始化数据预置，不得使用仅适用于 MySQL、PostgreSQL 或 Oracle<sup>[13]</sup>的专有语法。

### 2.2.3 标准规范

**接口规范：** 前端与后端、后端与 AI 模块之间的所有数据交互须遵循 RESTful 风格，采用 JSON 格式进行数据传输，所有接口须在《API 接口文档》中完整定义，包括请求方法、路径、参数格式、响应结构及错误码说明。

**数据安全规范：** 敏感数据的加密存储与脱敏展示须符合学校数据安全管理规定；传输层须采用 HTTPS 协议；身份认证须基于学号与微信账号的一对一实名绑定机制。

**权限管理规范：** 须严格实现基于角色的访问控制（RBAC）模型，四级角色（学院领导、管理老师、班团骨干、普通学生）的权限边界须在《数据库设计说明书》及代码注释中明确定义，不得存在权限越界的设计缺陷。

**文档规范：** 代码须具备完善的内联注释，关键业务逻辑须配有模块级说

明文档；交付物须包含《数据库设计说明书》、《API 接口文档》、《测试报告》及分角色《用户操作手册》，格式须符合软件工程课程项目的交付要求。

**外部数据交互规范：** 由于无法直接对接校级“微人大”系统接口，所有与外部系统的数据交互须通过 Excel、Word、PDF 等标准格式文件进行离线批量导入/导出，系统须对上传文件的格式合法性进行严格校验。

#### 2.2.4 开发工具

**版本控制：** 须使用 Git 进行源代码版本管理，以 Git 仓库链接作为最终代码成果的交付形式，确保代码变更历史完整可溯。

**数据库管理：** 须使用兼容金仓（Kingbase）的数据库客户端工具进行数据库结构设计、调试与维护，禁止直接在生产库上进行未经测试的结构变更。

**容器化部署工具：** 须支持 Docker 进行容器化打包，确保系统在学院 Linux 服务器上可跨平台、零配置直接运行，亦可交付 WAR/Jar 包配合 Tomcat 或 JVM<sup>[14]</sup>直接启动。

**接口调试工具：** 建议使用 Postman<sup>[15]</sup>或类似工具对 RESTful 接口进行调试与验证，确保《API 接口文档》与实际接口行为一致。

**原型设计工具：** UI 原型图须覆盖四级角色的核心操作路径，产出物须能清晰展示从学生提交申请到老师多级审批完成的完整交互流程。

#### 2.2.5 容量和性能要求

**并发容量：** 系统须支持日常约几十人同时在线的常规访问；在奖学金评

定、党团周报提交等高峰业务期，须向上兼容约 200 人的并发访问，总用户规模上限为 1200 名在院师生。

**响应时间：** 普通页面跳转及数据查询响应时间须控制在 1.5 秒以内；智能问答（AI 匹配）的首次响应时间不超过 3 秒；大规模 Excel/PDF 数据导入导出处理时间在网络正常情况下须在 10 秒内完成。

**资源占用：** 服务端须优化代码结构，确保在金仓数据库环境下高频 IO 操作不导致数据库死锁或内存溢出；数据库事务须合理设计以避免多端并发操作产生资源争用。

**数据存储：** 系统须持久化存储全院约 1200 名师生的基本信息、党团流程数据、成绩数据及证明文件，数据库设计须预留充足的存储余量，并支持历史数据归档。

#### 2.2.6 灵活性和配置要求

**模块化扩展性：** 系统架构须支持在不影响现有功能的前提下，快速新增业务模块（如出国申请、评奖评优等）。各模块须通过标准接口与核心服务交互，新模块的接入不应要求对现有数据库表结构或核心业务逻辑进行侵入性修改。

**知识库可配置性：** 政策知识库须支持管理员通过后台界面进行内容的自助维护（上传、修改、删除政策文件），无需开发人员介入即可完成知识库更新，确保系统内容随学院政策变化保持时效性。

**审批流可配置性：** 电子审批流程须支持在 1 至 2 天内执行撤销或重批操

作，以应对审批信息填写有误或审批决策需要变更的实际场景。

**数据导入模板可配置性：** 系统须支持管理员自定义或更新 Excel/PDF 导入模板，以适应学院不同时期数据格式的调整需求；对于不符合模板规范的上传文件，须提供精准的逐条错误定位反馈。

**通知渠道可配置性：** 信息推送模块须支持系统内消息与邮件等多种通知渠道，管理员须能够根据通知类型和紧急程度自由选择发送方式，以提高信息触达率。

### 3、 软件设计

本章所给出的各类图示用于描述系统的静态结构与动态行为。其中，体系结构图与包图用于说明系统的整体组成；页面跳转图与界面原型用于说明用户交互方式；顺序图用于描述核心业务流程的实现过程；类图和 E-R 图用于说明系统的数据与对象结构；部署图用于说明目标运行环境及各节点之间的连接关系。

#### 3.1 软件体系结构设计

本系统采用前后端分离的多端协同架构。学生用户主要通过微信小程序访问系统，完成政策咨询、党团进度查询、证明申请、成绩单上传与学业分析等操作；管理员、老师及助教主要通过 PC Web 后台访问系统，完成政策知识库维护、审批处理、信息发布、学生数据管理与日志审计等操作。

服务端按照业务职责划分为认证与权限服务、用户与学生画像服务、智能问答服务、党团流程服务、信息推送服务、电子证明与审批服务、学业分

析服务、文件解析服务、PDF 生成服务和日志审计服务。数据层采用 Kingbase 数据库保存结构化业务数据，使用文件存储保存政策文件、模板文件、成绩单和生成后的 PDF 文件。该架构能够满足系统对模块化、可维护性、数据安全和后续扩展的要求。

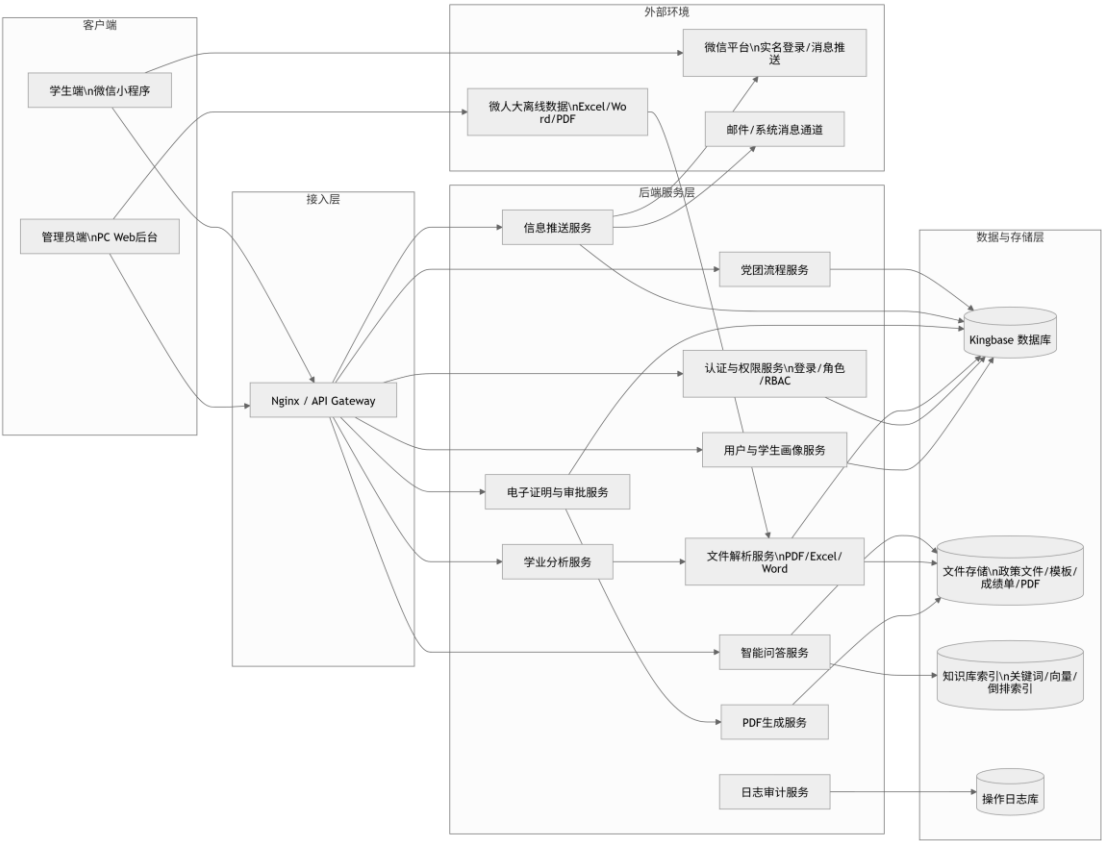


图 3-1 系统总体架构图

图 3-1 从整体上展示了系统各端、各服务和数据存储之间的关系。学生端和管理端均通过统一的 API 接入后端服务，避免前端直接访问数据库或文件存储。后端业务服务负责封装核心业务逻辑，并通过统一的数据访问层读写 Kingbase 数据库。

其中，智能问答服务主要依赖政策知识库和检索索引；学业分析服务依赖成绩单解析模块和培养方案数据；审批服务依赖审批流状态、PDF 模板和

用户权限数据；日志审计服务对涉及审批、敏感信息查看、数据导入导出等关键操作进行记录。通过这种分层结构，可以降低不同模块之间的耦合度，便于后续独立测试与迭代。

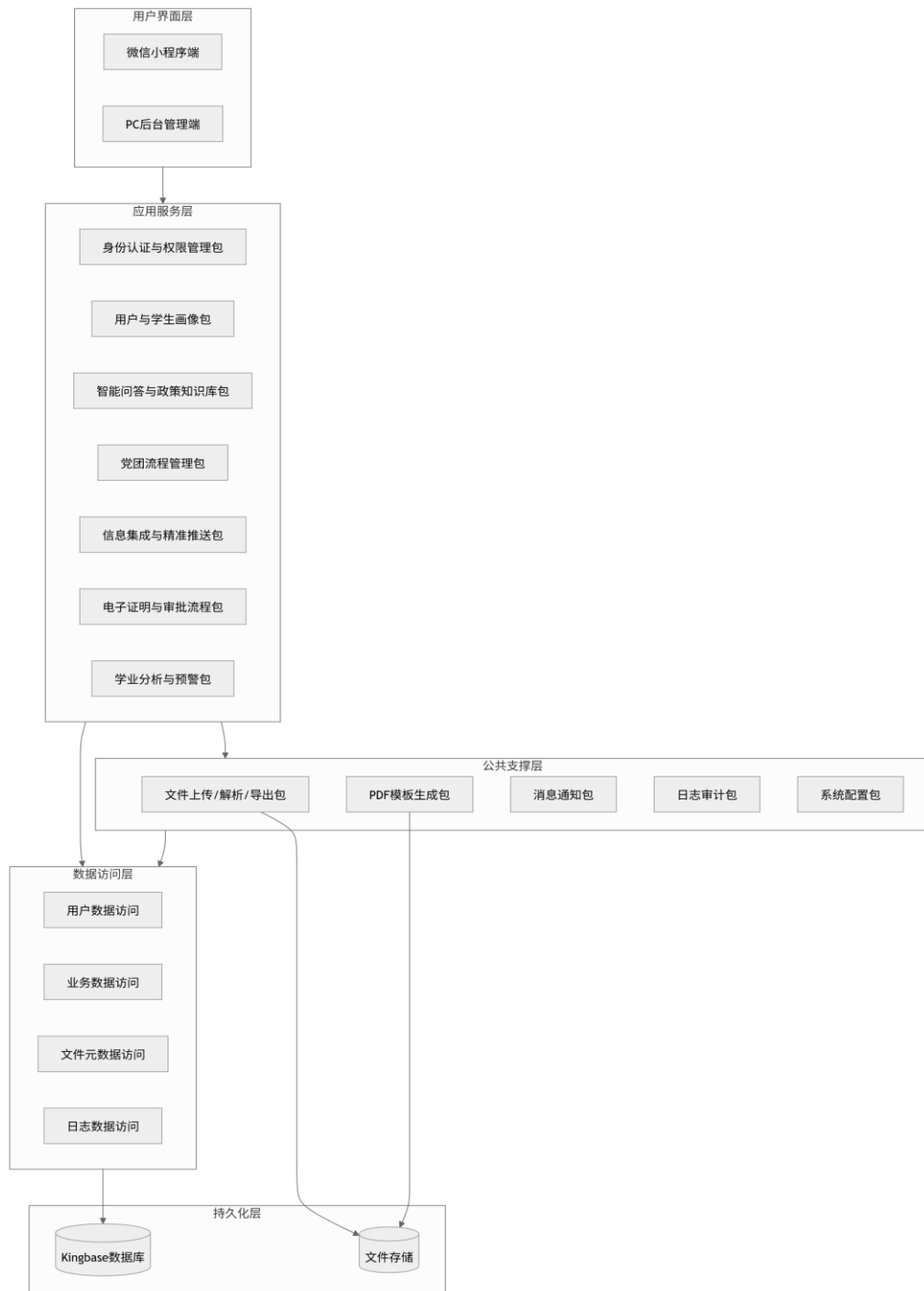


图 3-2 模块划分图

图 3-2 对系统进行了逻辑包划分。用户界面层负责与用户交互，不直接承担复杂业务判断；应用服务层负责处理各模块核心业务；公共支撑层负责提供文件处理、PDF 生成、消息通知和日志记录等通用能力；数据访问层负责屏蔽具体数据库操作细节。

这种包结构能够支持并行开发：前端人员可以围绕小程序端和 PC 后台端实现页面与交互；后端人员可以分别实现用户权限、审批、党团流程、学业分析等服务；数据库人员可以根据数据访问层和 E-R 图完成表结构设计；AI 模块人员可以专注于知识库检索与问答准确性控制。

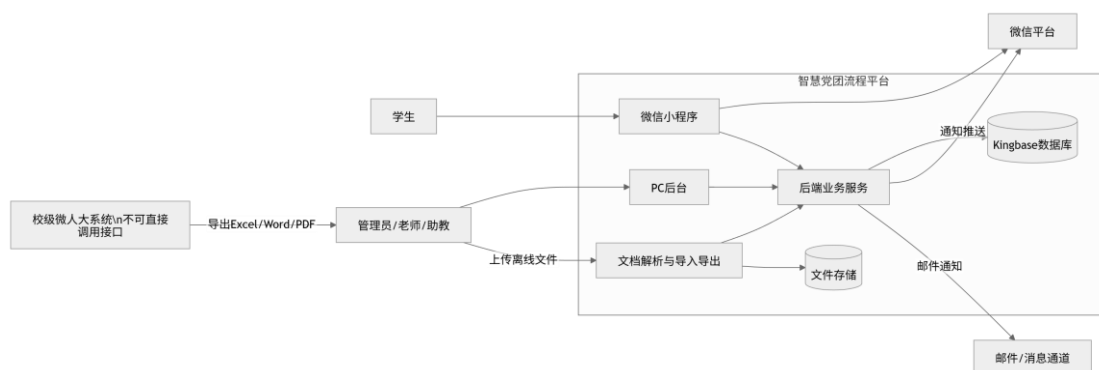


图 3-3 外部系统与数据交互图

图 3 说明了本系统与外部环境之间的数据交互关系。由于系统无法直接调用校级“微人大”系统接口，因此与外部系统之间的数据流转主要采用 Excel、Word、PDF 等离线文件导入导出方式完成。该设计虽然降低了实时同步能力，但能够在现有条件下保证系统可实现性和数据流转的可控性。

对于外部导入的数据，系统需要进行格式校验、字段合法性校验和错误定位提示；对于导出的数据，系统需要根据用户角色进行权限校验和敏感字

段脱敏。所有导入、导出、审批和敏感信息查看操作都应写入审计日志，以满足系统对可追溯性的要求。Part1 中也明确提出了敏感数据加密、四级权限控制和审计追踪等约束。

### 3.2 用户界面设计

用户界面设计遵循“严谨、正式、简约”的原则。系统面向学生、老师、助教和学院管理人员，界面风格应避免过度娱乐化，重点突出事务办理效率、信息层级清晰和操作反馈明确。

学生端界面采用移动端常见的底部标签导航结构，将高频功能集中在首页和底部导航栏中，保证政策咨询、党团进度、证明申请和学业分析等核心业务能够在较少点击次数内完成。管理员端界面采用“左侧导航 + 顶部状态栏 + 右侧内容区”的后台管理布局，方便管理员进行批量审批、数据导入导出、信息发布和统计分析。Part1 中已明确 PC 端与移动端的界面布局原则，以及智能问答、党团流程、学生画像和学业分析看板等核心界面要求

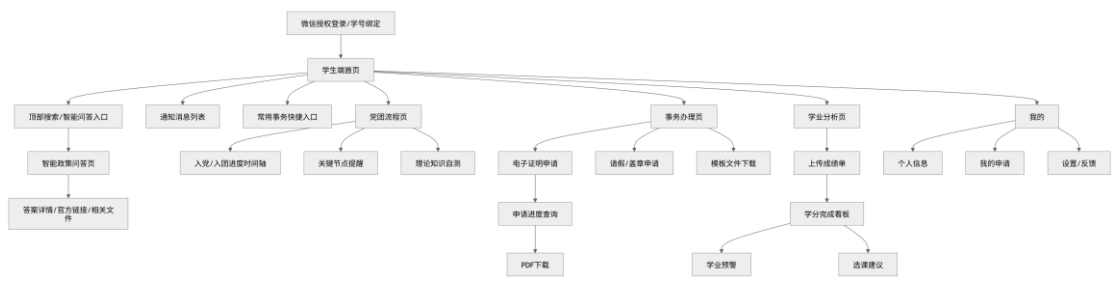


图 3-4 展示了学生端主要页面之间的跳转关系。学生登录后进入首页，首页提供智能问答、党团流程、事务办理、学业分析和个人中心等入口。对于学生高频使用的功能，如政策查询、证明申请和进度查询，界面应尽量减

少层级，原则上不超过三次点击即可完成主要操作。

在交互设计上，学生端需要突出“待办提醒”和“状态反馈”。例如，证明申请提交后应立即显示申请编号和当前状态；党团流程页面应展示当前阶段、已完成节点和待完成任务；学业分析页面应通过颜色、标签或进度条区分已完成学分模块与存在风险的模块。

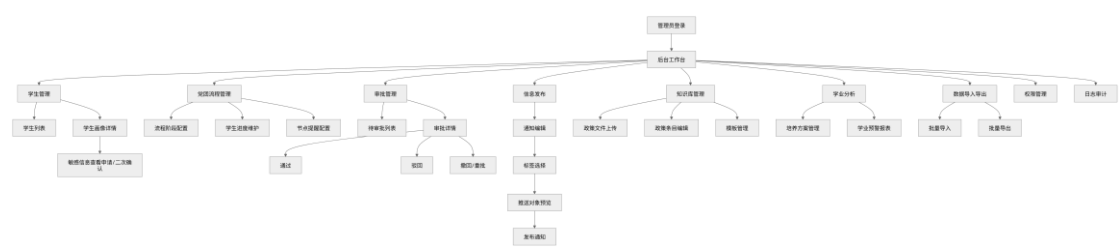


图 3-5 管理端页面跳转图

图 3-5 展示了管理员端主要页面之间的跳转关系。管理员端的设计重点是批量处理和权限区分，因此需要提供清晰的左侧导航栏，并按照业务模块划分为工作台、学生管理、党团流程管理、审批管理、信息发布、知识库管理、学业分析、数据导入导出、权限管理和日志审计等页面。

管理端的关键操作应提供二次确认和结果反馈。例如，审批通过、审批驳回、导出敏感数据、查看学生敏感信息、批量导入文件等操作均应弹出确认框，并在操作完成后给出成功、失败或部分失败的详细提示。对于批量导入失败的情况，系统应返回具体错误行、错误字段和修改建议，避免管理员难以定位问题。



图 3-6 学生端核心界面原型-问答页



图 3-7 学生端核心界面原型-党团发展进度页

图 3-6、图 3-7 给出了学生端核心界面的原型设计。智能问答页面采用对话式交互方式，顶部提供高频问题标签，底部提供问题输入框；党团流程页面采用线性时间轴展示入党、入团流程阶段，并对当前阶段进行明显标注。



图 3-8 管理端核心界面原型-审批页

图 3-8 给出了管理端核心界面的原型设计。管理端界面需要根据角色动态控制可见菜单和可操作按钮。学院领导、管理老师、班团骨干和普通学生具有不同权限范围，系统不应仅在前端隐藏按钮，还应在后端进行二次权限校验，防止越权访问。

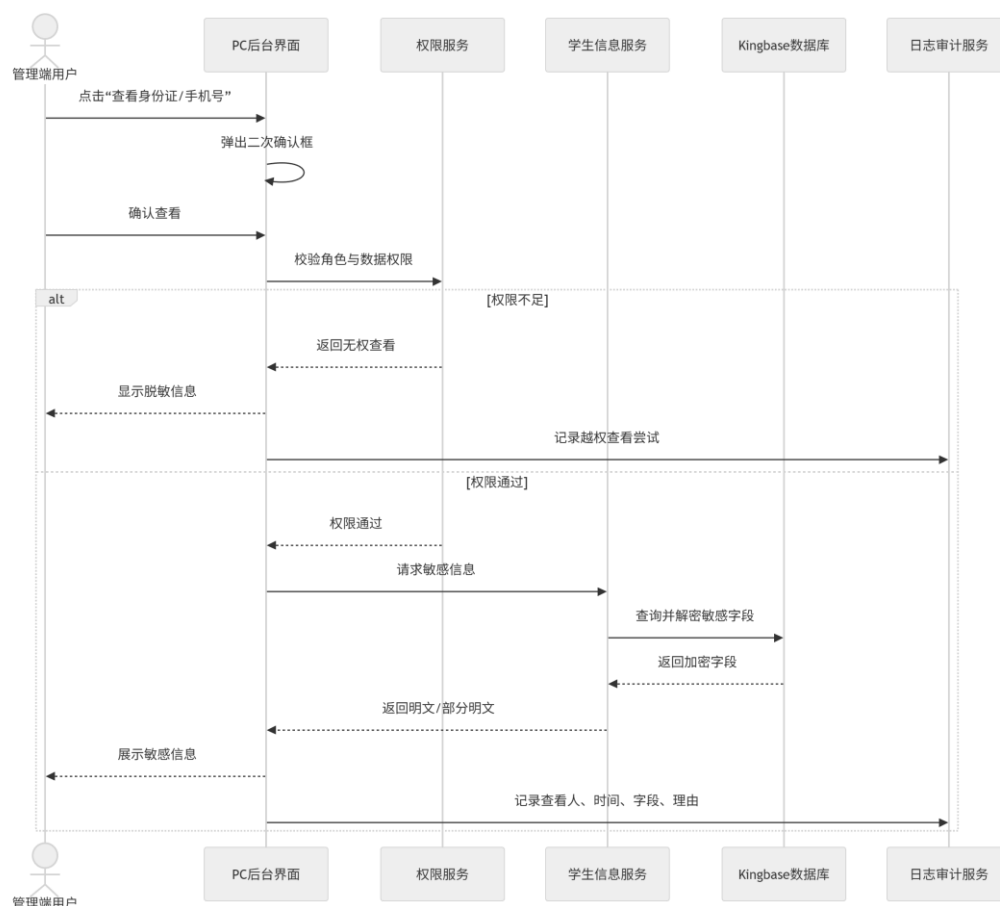


图 3-9 敏感信息查看确认流程图

图 3-9 展示了敏感信息查看时的交互过程。系统默认对身份证号、手机号、生源地、处分记录等敏感字段进行脱敏展示。只有具有相应权限的用户在通过二次确认后，才能查看完整信息。

敏感信息查看不应被视为普通查询操作，而应作为高风险操作处理。系统需要记录查看人、查看时间、查看对象、查看字段和操作来源。对于权限不足的用户，系统仅返回脱敏信息，并记录越权查看尝试。该设计能够同时满足业务管理需要和学生隐私保护要求。

3.3 用例设计

本节对系统核心用例的实现过程进行设计说明。需求规格说明书中已经列出了 18 个功能点，本设计说明书选取其中最具代表性的核心业务流程进行详细建模，包括智能政策问答、政策知识库维护、党团流程进度查询、信息精准推送、电子证明申请与审批、成绩单解析与学业分析。

各用例顺序图采用“用户界面层—控制器层—服务层—数据层/外部模块”的结构进行描述。该结构能够清晰展示一次用户操作从前端提交请求，到后端执行业务逻辑，再到数据库读写和结果返回的完整过程。

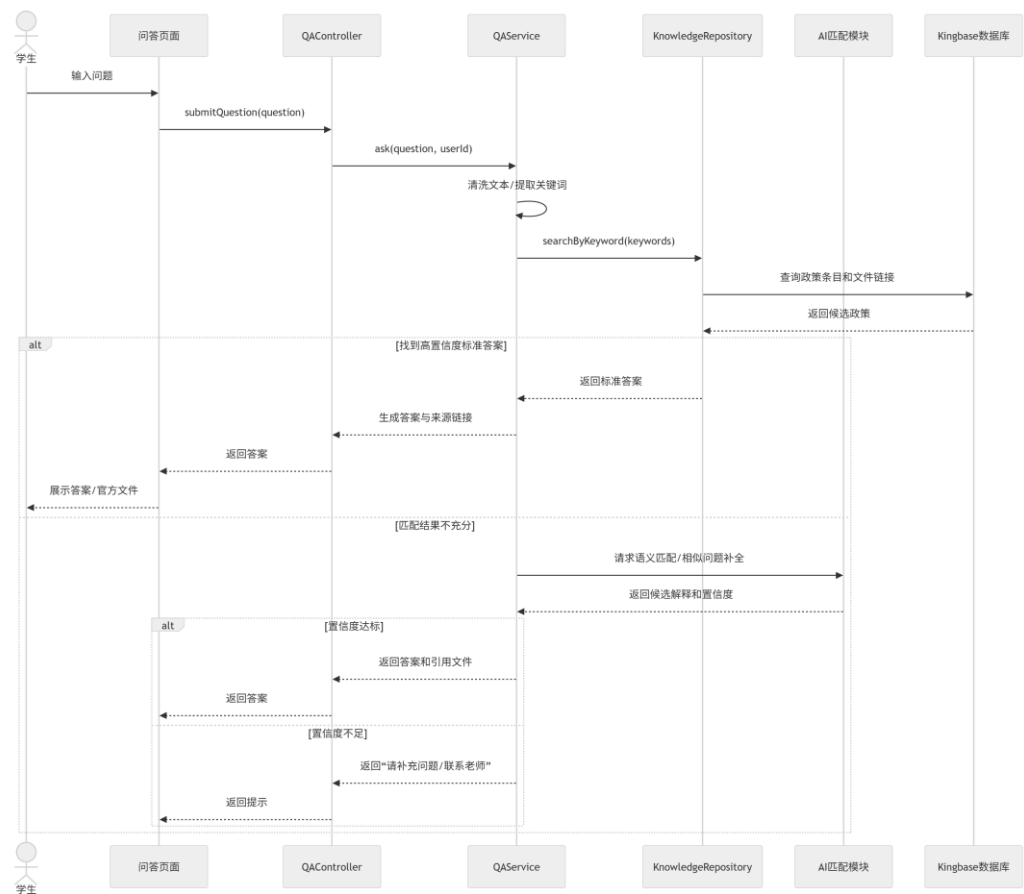


图 3-10 智能政策问答顺序图

图 3-10 描述了学生使用智能政策问答功能的实现过程。学生在问答页面输入自

然语言问题后，系统首先进行文本清洗和关键词提取，然后检索政策知识库。若系统能够找到高置信度的标准答案，则直接返回答案和官方文件来源；若匹配结果不足，则调用语义匹配或 AI 辅助模块进行补充判断。

为避免问答模块产生不可靠回答，系统应优先返回知识库中的标准答案、政策文件原文或官方链接。当置信度不足时，系统不应强行生成答案，而应提示用户补充问题或联系相关老师。Part1 中已经明确要求智能问答优先基于标准答案或官方文件链接，避免大模型“幻觉”。

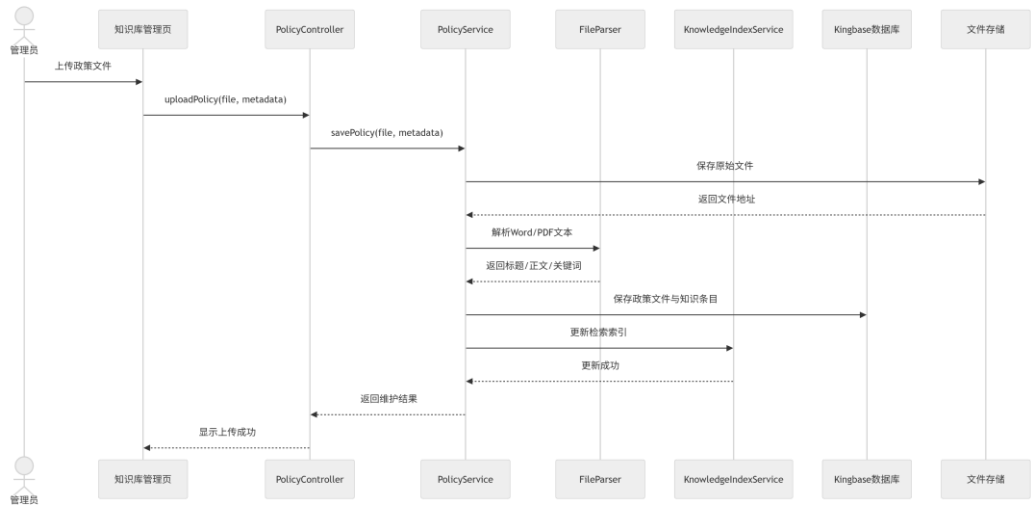


图 3-11 政策知识库维护顺序图

图 11 描述了管理员维护政策知识库的过程。管理员上传政策文件后，系统将原始文件保存至文件存储，并调用文件解析模块提取标题、正文、关键词和可检索片段。解析完成后，系统将政策文件元数据和知识条目写入数据库，并更新检索索引。

知识库维护功能需要支持文件上传、文件更新、条目编辑、条目删除和版本记录。对于已经被问答模块引用的政策条目，系统不应直接物理删除，而应采用逻辑删除或版本替换方式，保证历史问答记录仍然可追溯。

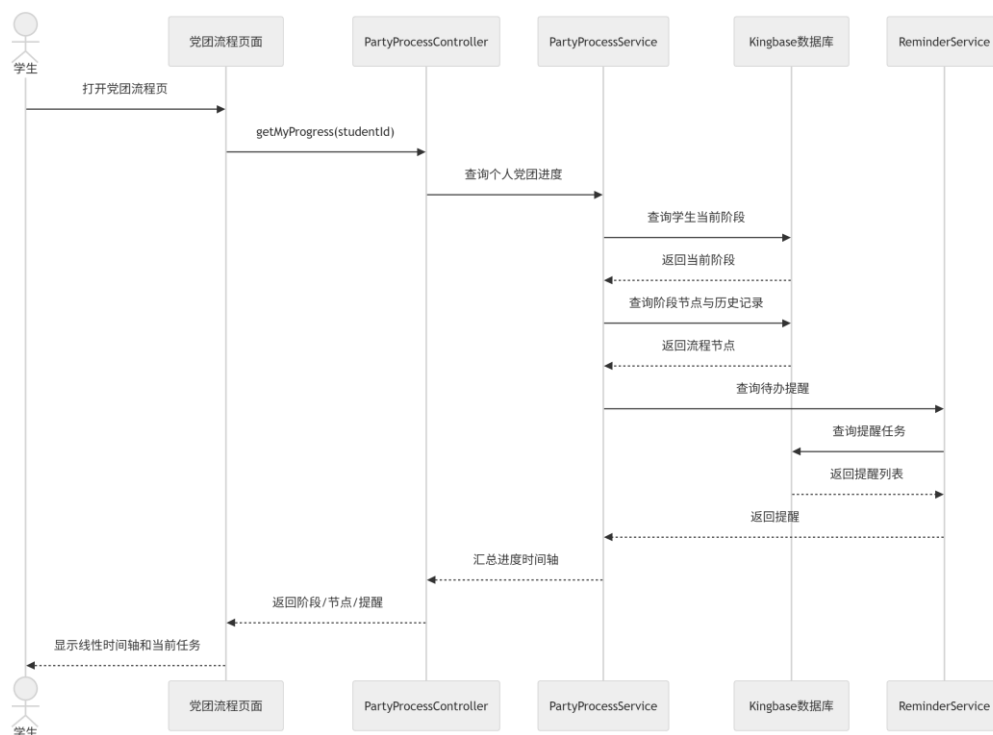


图 3-12 党团流程进度查询顺序图

图 3-12 描述了学生查询个人党团流程进度的过程。学生进入党团流程页面后，系统根据学生身份查询其当前流程类型、当前阶段、历史完成节点和待办提醒，并将结果以时间轴形式返回前端。

党团流程模块的设计重点是流程状态的一致性。学生端只负责查看个人进度和接收提醒，流程阶段的维护和调整应由具备权限的管理员完成。对于关键节点提醒，系统应根据流程阶段、时间规则和学生当前状态自动生成待办事项，避免学生遗漏材料提交、思想汇报、党课学习等关键任务。

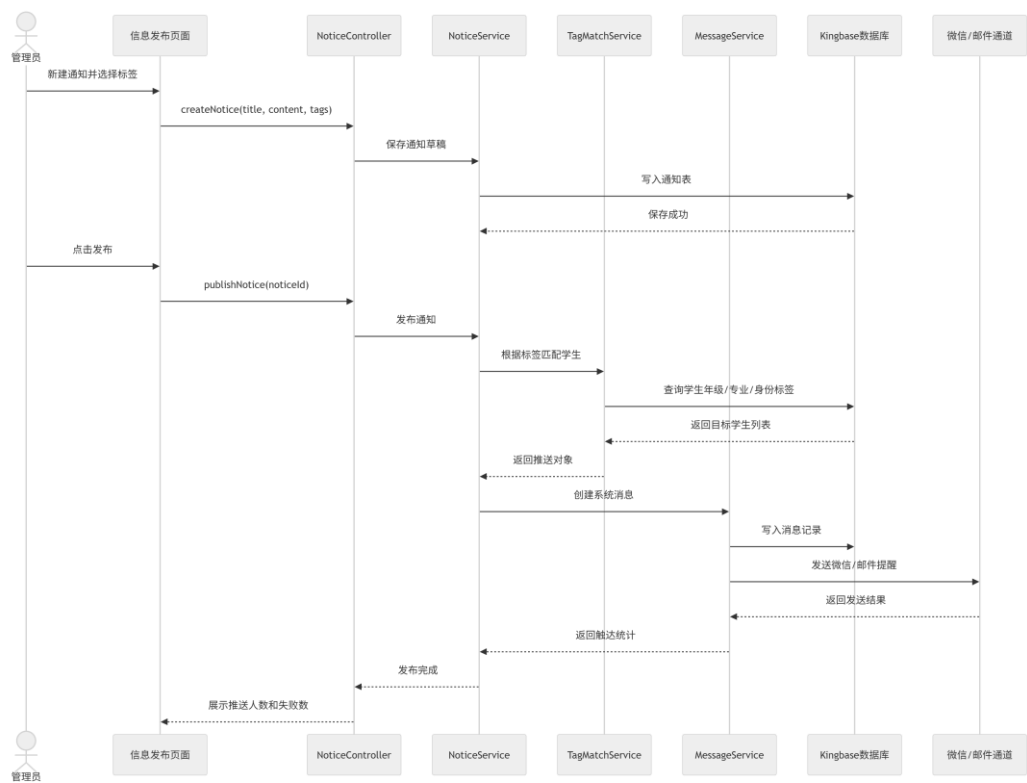


图 3-13 信息精准推送顺序图

图 3-13 描述了信息精准推送的实现过程。管理员创建通知后，需要为通知设置标签，例如年级、专业、就业、实习、竞赛、党团等。系统根据通知标签与学生画像信息进行匹配，生成目标接收人列表，并通过系统消息、微信消息或邮件进行发送。

精准推送模块应支持发送前预览目标群体，避免误发或漏发。通知发布后，系统需要记录发送人数、成功人数、失败人数和阅读状态，供管理员后续查看信息触达效果。对于涉及敏感范围或重要事项的通知，应保留发布人、发布时间和修改记录。

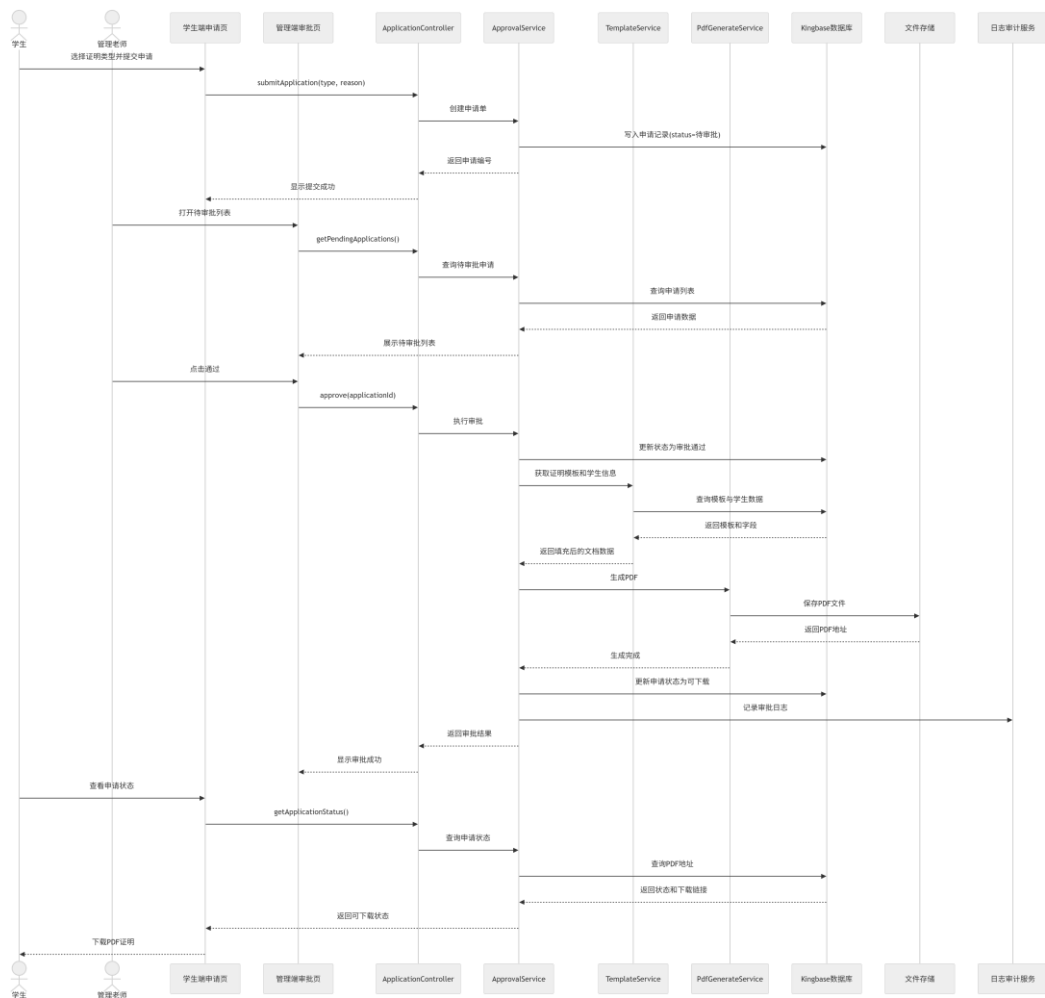


图 3-14 电子证明申请与审批顺序图

图 3-14 描述了电子证明申请与审批的完整流程。学生提交证明申请后，系统创建申请记录并设置为“待审批”状态。管理员在后台查看申请详情后，可以执行通过、驳回或退回修改等操作。审批通过后，系统根据证明模板和学生信息自动生成 PDF 文件，并将申请状态更新为“可下载”。

审批流程模块需要保证状态转换的严谨性。申请从创建到完成，应经历“已提交—待审批—审批中—审批通过/审批驳回—可下载/已结束”等状态。对于误操作，系统应支持在规定时间内撤回或重批；对于已经生成的 PDF 文件，应保存生成时间、模板版本和审批记录，保证证明文件来源可追溯。

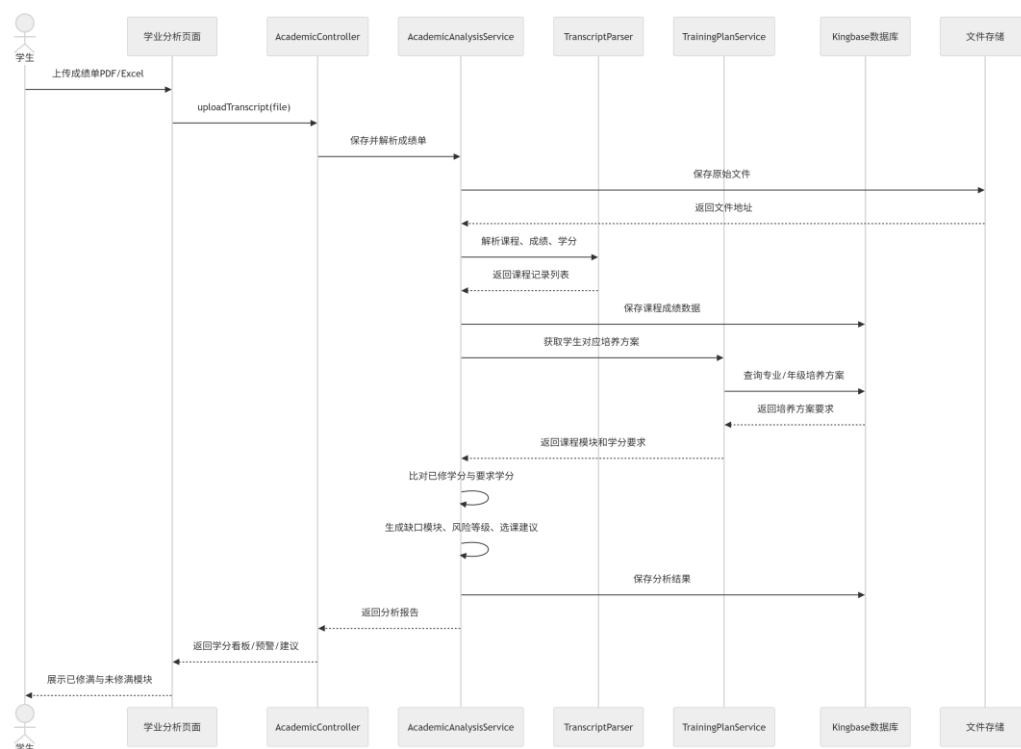


图 3-15 成绩单解析与学业分析顺序图

图 3-15 描述了成绩单解析与学业分析的实现过程。学生上传 PDF 或 Excel 成绩单后，系统首先保存原始文件，然后调用成绩单解析模块提取课程名称、成绩、学分和课程类别等信息。解析结果写入数据库后，系统根据学生所在年级和专业读取对应培养方案，并进行学分模块比对。

学业分析模块的输出结果应包括已完成学分、缺口学分、未完成模块、风险等级和选课建议。由于成绩单解析可能受到文件格式影响，系统应在解析失败时给出明确错误提示，并允许学生重新上传或进行人工确认。对于培养方案数据，系统应支持按年级、专业和版本进行维护，避免不同培养方案混用。

### 3.4 类设计

本节对系统核心领域对象和关键业务模块进行类设计。类设计以业务对象为中

心，结合控制器、服务类和数据实体类进行组织。为了保持系统结构清晰，业务逻辑主要封装在 Service 类中，Controller 只负责接收请求和返回响应，Entity 类用于表示数据库中的持久化对象，DTO/VO 类用于前后端数据传输和展示。

系统类设计遵循高内聚、低耦合原则。不同业务模块之间通过服务接口进行调用，避免直接访问对方内部数据结构。例如，审批服务需要学生信息时，应通过学生信息服务获取，而不是直接操作学生表；学业分析服务需要培养方案时，应通过培养方案服务读取对应版本数据。

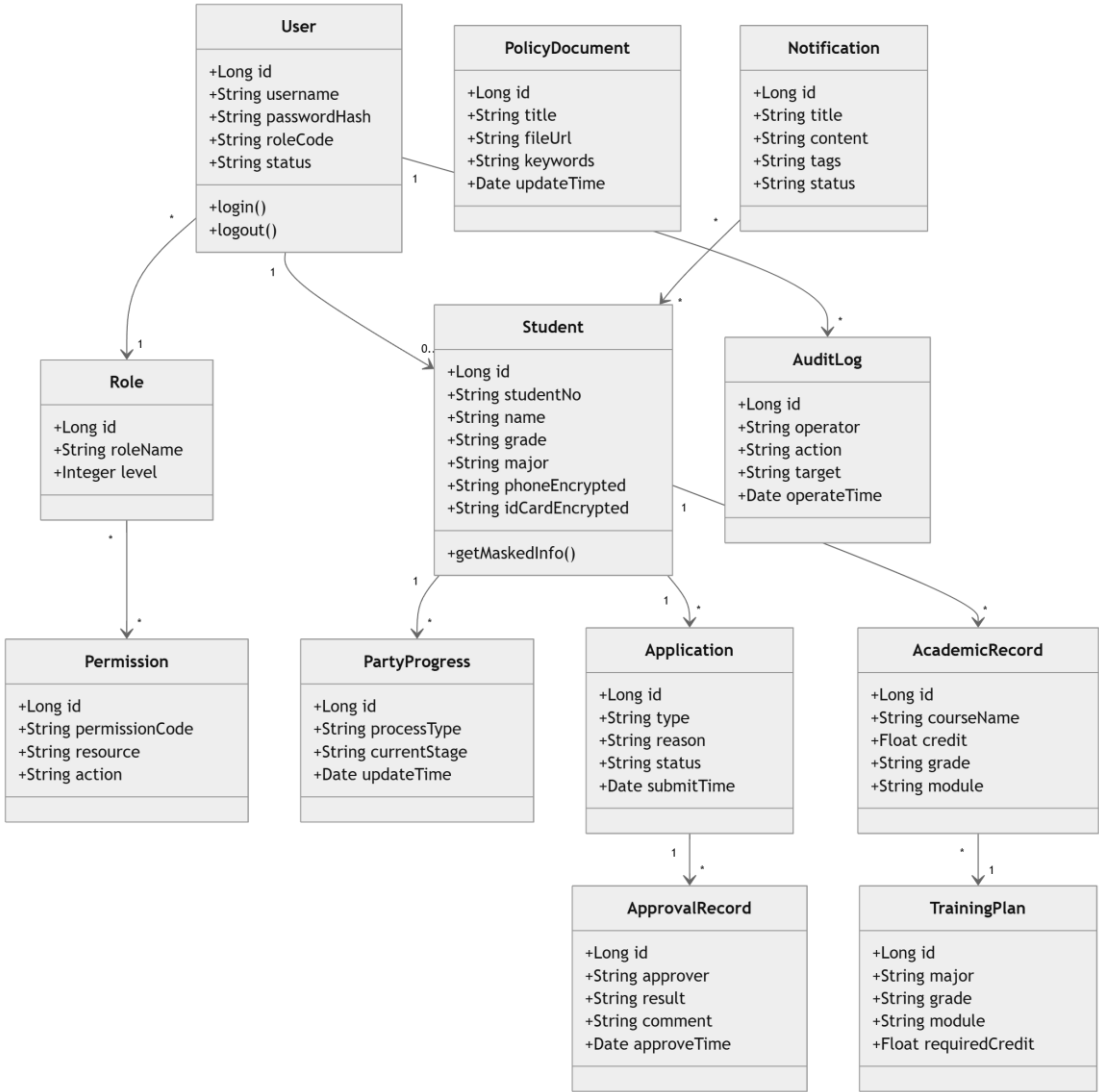


图 3-16 系统核心领域类图

图 3-16 展示了系统的核心领域类及其关系。User 类表示系统登录用户, Student 类表示学生业务信息, Role 和 Permission 类用于实现角色权限控制。Application 和 ApprovalRecord 用于表示事务申请和审批记录, PolicyDocument 用于表示政策文件, Notification 用于表示通知信息, AcademicRecord 和 TrainingPlan 用于表示学生成绩与培养方案。

在实现时, User 与 Student 不应完全合并。User 主要负责登录认证、角色绑定和账号状态管理; Student 主要负责学生学号、年级、专业、联系方式和学业信息等业务属性。这样可以兼容管理员、老师、助教等非学生用户, 也便于后续扩展多种用户身份。

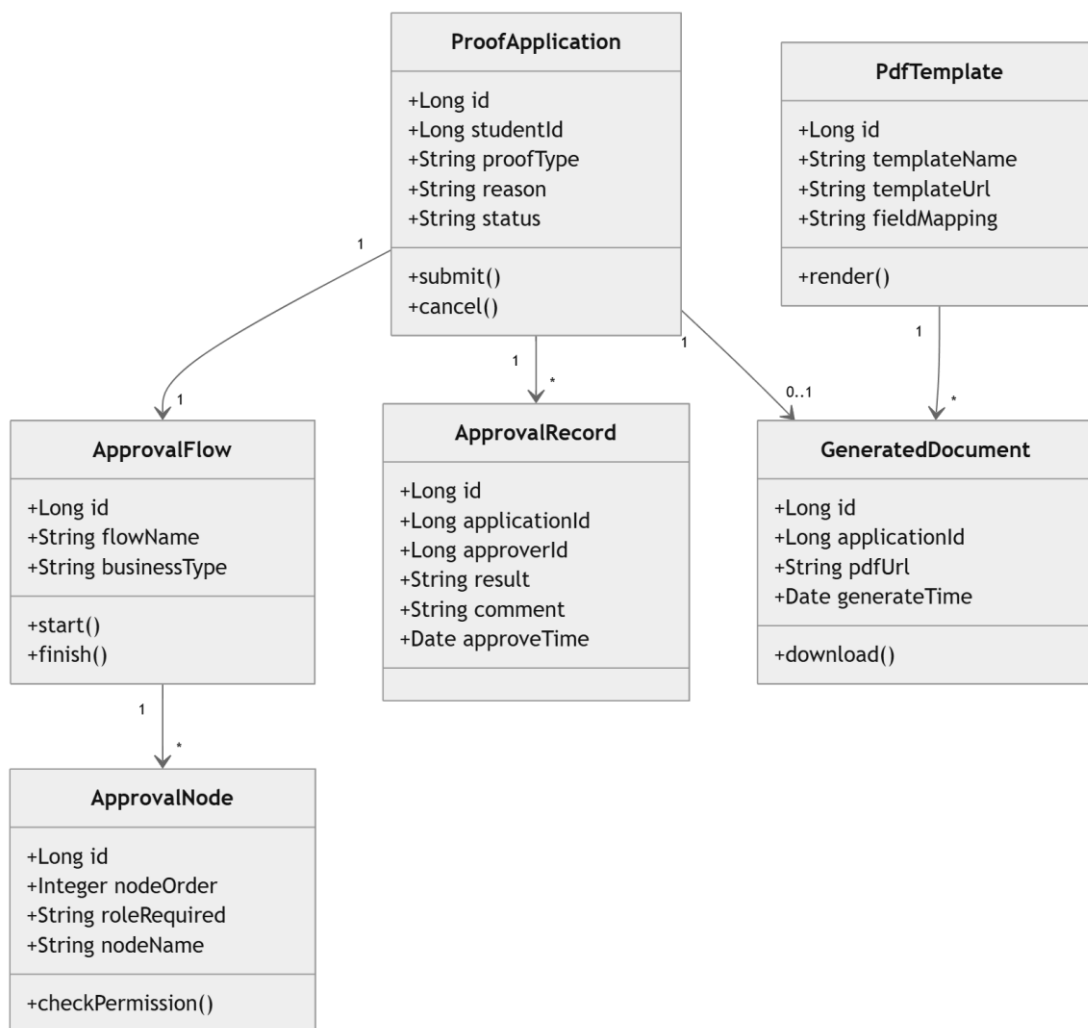


图 3-17 审批与证明模块设计类图

图 3-17 展示了审批与证明模块的主要类结构。ProofApplication 表示学生提交的证明申请，ApprovalFlow 表示审批流程，ApprovalNode 表示审批节点，ApprovalRecord 表示每次审批操作记录，PdfTemplate 表示证明模板，GeneratedDocument 表示最终生成的 PDF 文件。

该模块的设计重点是审批状态和审批记录的分离。申请对象保存当前状态，审批记录保存历史操作。这样既能快速查询申请当前进度，也能完整追溯每一次审批动作。PDF 文件生成不应与审批操作强耦合，审批通过后由 PDF 生成服务根据模板和学生数据生成文件，便于后续替换模板或增加新的证明类型。

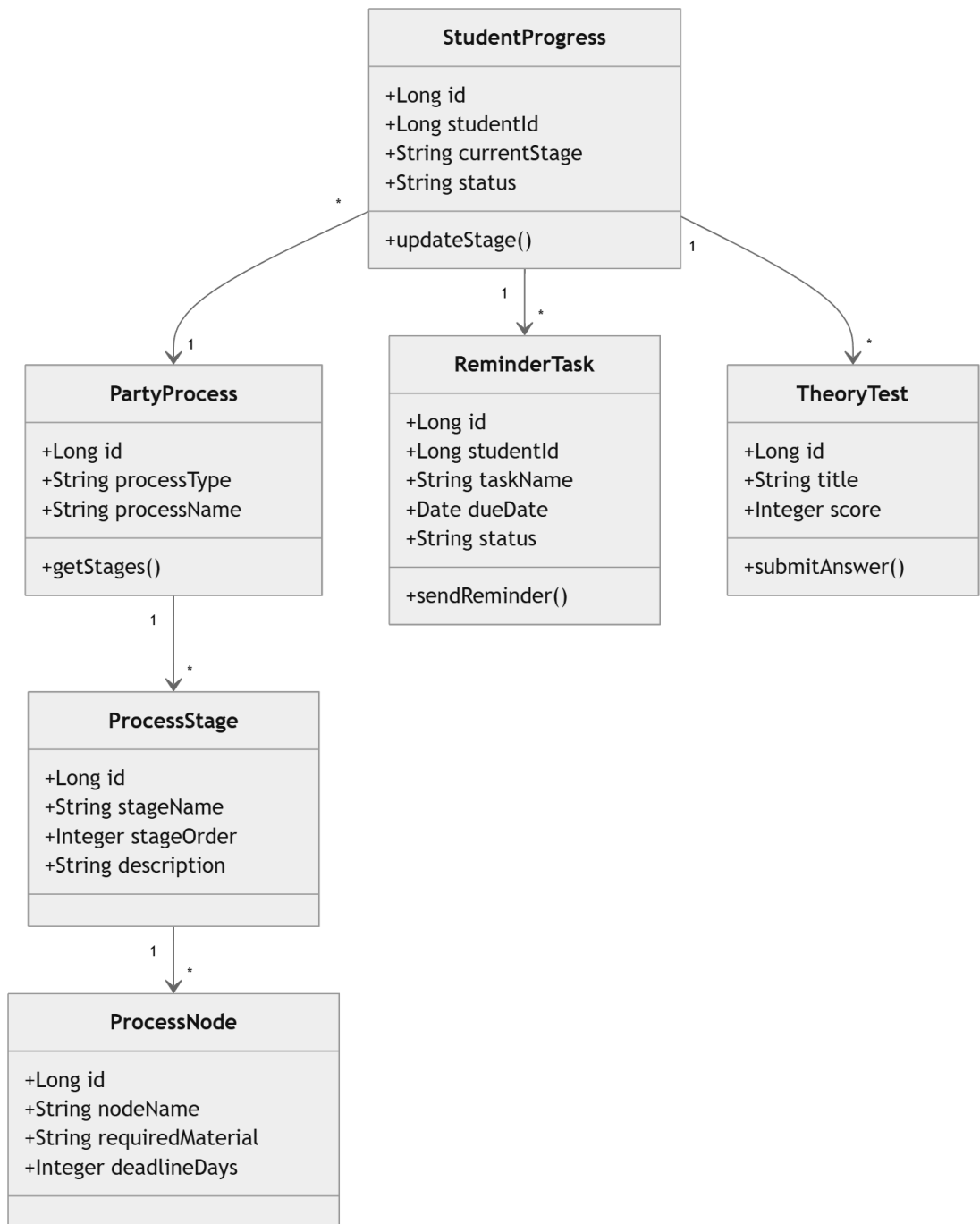


图 3-18 党团流程模块设计类图

图 3-18 展示了党团流程模块的主要类结构。PartyProcess 表示入党或入团流程，ProcessStage 表示流程阶段，ProcessNode 表示阶段中的具体节点，StudentProgress 表示学生个人进度，ReminderTask 表示关键节点提醒，

TheoryTest 表示理论知识自测。

党团流程模块应将“标准流程配置”和“学生个人进度”分开存储。标准流程由管理员维护，表示流程应当如何进行；学生个人进度表示某个学生当前走到哪一步。这样可以避免修改标准流程时破坏历史数据，也便于为不同年级、不同身份学生配置不同流程。

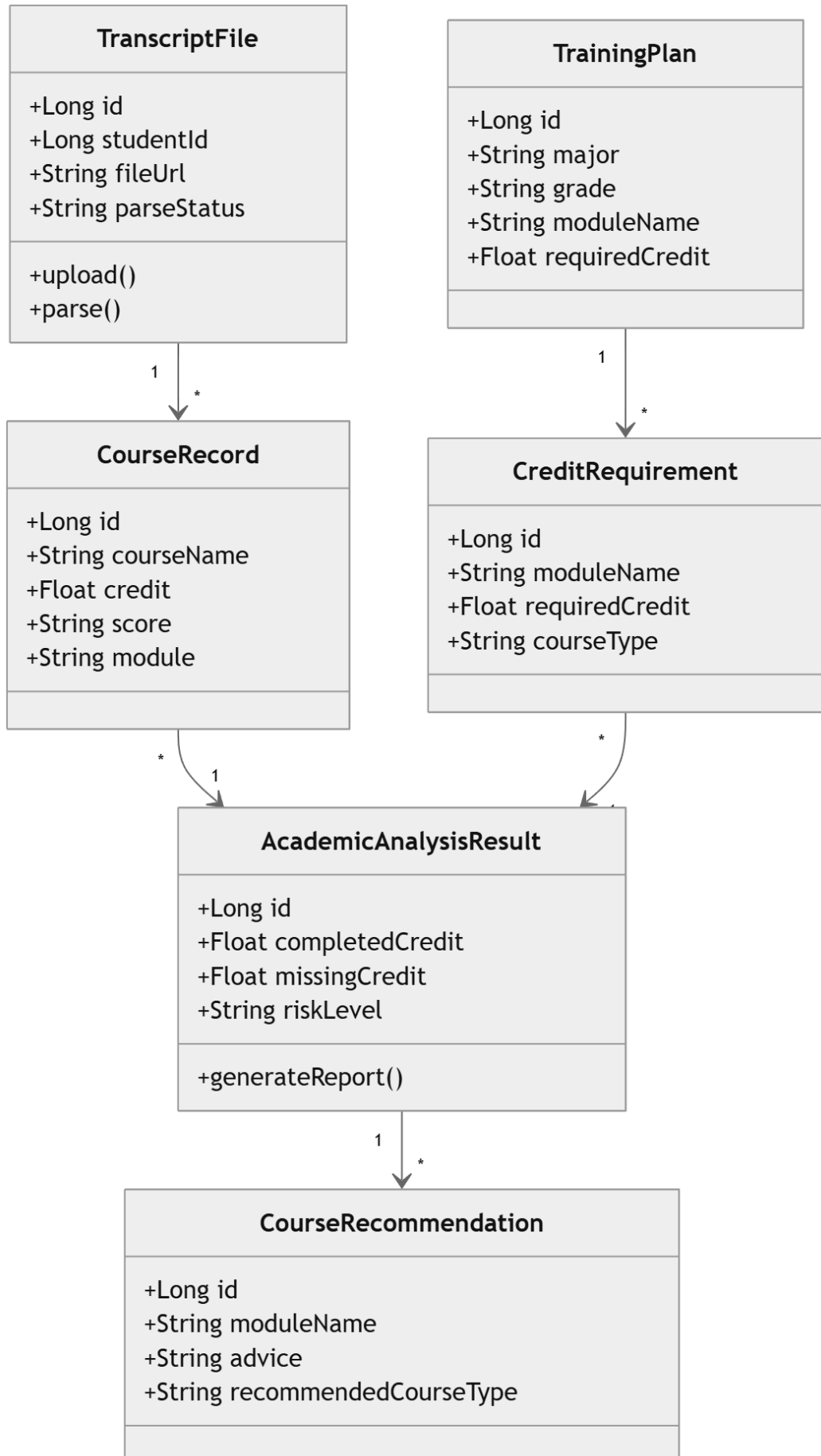


图 3-19 学业分析模块设计类图

图 3-19 展示了学业分析模块的主要类结构。TranscriptFile 表示学生上传的成绩单文件，CourseRecord 表示解析后的课程记录，TrainingPlan 和 CreditRequirement 表示培养方案及学分要求，AcademicAnalysisResult 表示分析结果，CourseRecommendation 表示选课建议。

学业分析模块需要特别注意数据来源和分析结果的可解释性。系统不应只给出“是否预警”的结论，还应展示造成预警的具体原因，例如某模块缺少多少学分、哪些必修课未完成、哪些课程类别需要补足。这样可以帮助学生理解分析结果，也方便管理员核查系统判断是否正确。

### 3.5 数据设计

本节对系统中的永久数据进行设计说明。系统的主要持久化数据包括用户与角色权限数据、学生基础信息、政策知识库数据、党团流程数据、通知推送数据、事务申请与审批数据、成绩与培养方案数据、文件元数据和操作审计日志。

数据库采用 Kingbase 作为主要存储系统。结构化业务数据保存于数据库中，政策文件、模板文件、学生上传文件和生成后的 PDF 文件保存于文件存储中，数据库中仅保存文件编号、文件路径、文件类型、上传人和上传时间等元数据。该方式可以避免数据库存储大量二进制文件，提高系统维护性和查询效率。

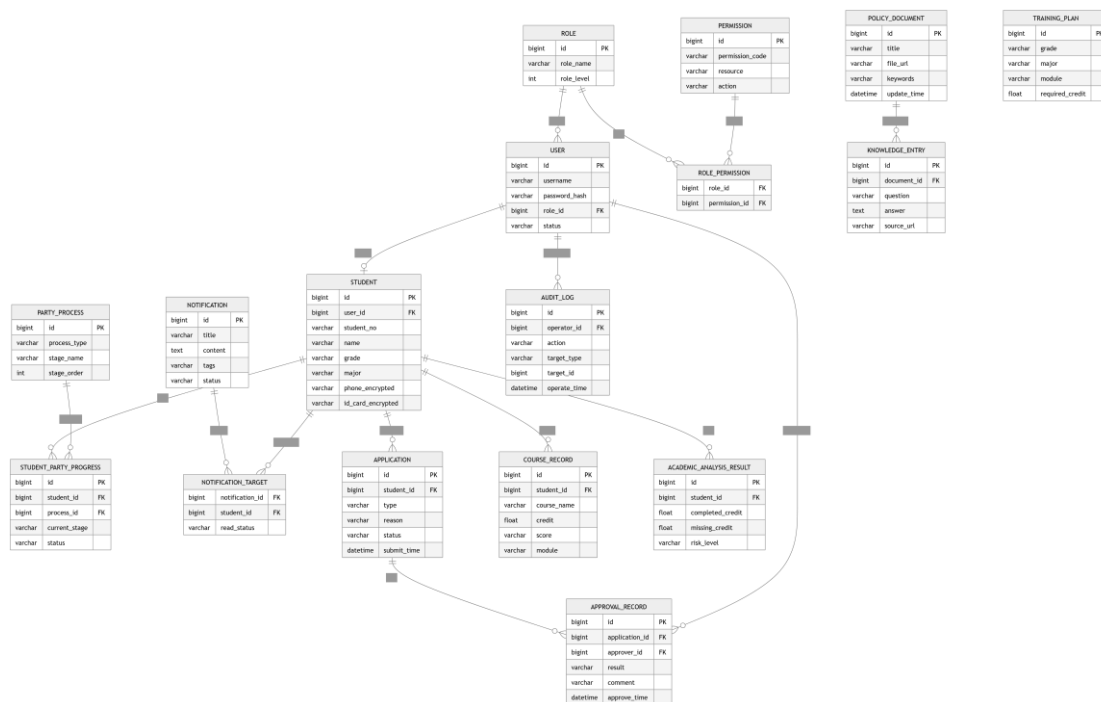


图 3-20 数据库 E-R 图

图 3-20 展示了系统主要数据表及其关系。用户表、角色表和权限表用于支持登录认证与分级权限控制；学生表保存学生基础信息；政策文件表和知识条目表支持智能问答；党团流程表和学生进度表支持流程可视化；通知表和通知接收表支持精准推送；申请表和审批记录表支持电子审批；课程记录表、培养方案表和学业分析结果表支持学业预警。

对于身份证号、手机号等敏感字段，数据库中应采用加密存储，不应保存明文。

前端展示时应根据角色进行动态脱敏。对于审批记录、导入导出记录和敏感信息查看记录，应采用追加写入方式保存，不建议随意删除，以保证系统审计的完整性。

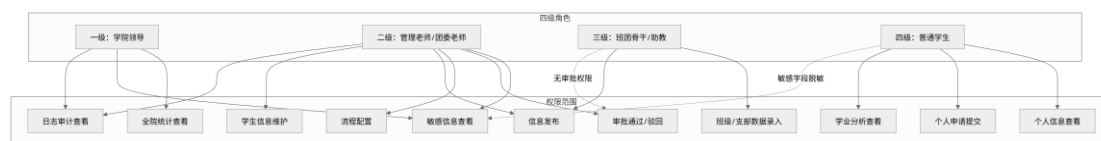


图 3-21 四级权限 RBAC 模型图

图 21 展示了系统的四级权限模型。系统角色分为学院领导、管理老师、班团骨干和普通学生。学院领导侧重统计查看和审计监督；管理老师负责审批、流程配置、信息发布和学生数据维护；班团骨干可以承担部分数据录入和信息协助工作，但不应拥有正式审批权限；普通学生仅能查看和维护本人相关信息。

权限控制应同时体现在菜单权限、按钮权限、数据权限和接口权限四个层面。仅在前端隐藏菜单是不充分的，后端接口必须重新校验用户角色和操作权限。对于敏感数据，即使用户能够访问学生信息页面，也应根据其权限决定展示明文、部分脱敏信息或完全不可见信息。

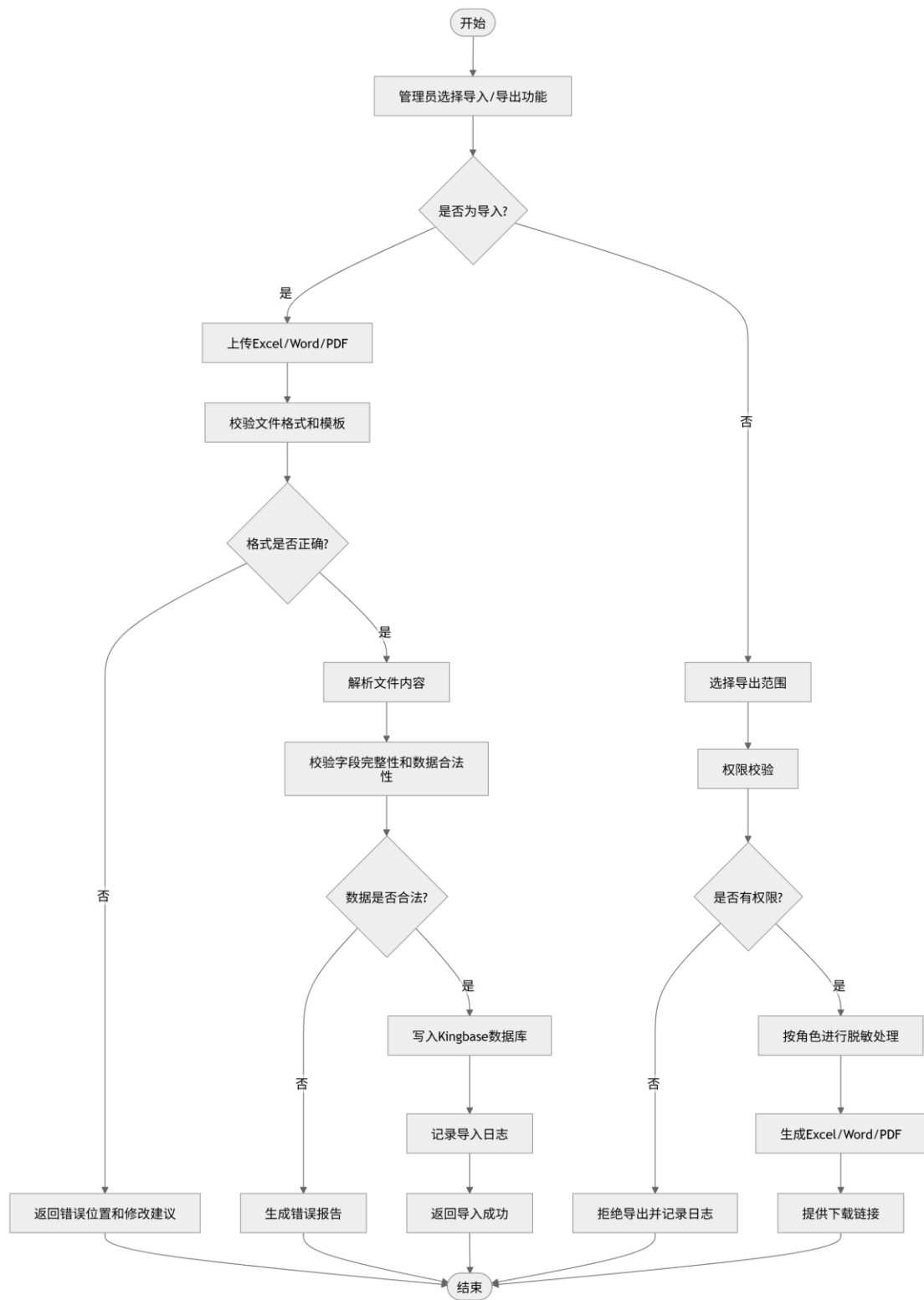


图 3-22 数据导入导出活动图

图 3-22 展示了数据导入导出的处理过程。导入流程包括文件上传、格式校验、内容解析、字段合法性检查、数据库写入和导入日志记录。若导入失败，系统应

给出错误行号、错误字段和失败原因，帮助管理员快速修正文件。

导出流程需要先进行权限校验，再根据角色进行数据脱敏处理，最后生成 Excel、Word 或 PDF 文件。涉及学生敏感信息的导出应视为高风险操作，系统需要弹出二次确认，并记录导出人、导出时间、导出范围和文件类型。该设计可以降低批量数据泄露风险。

### **3.6 部署设计**

本系统部署于 Linux 服务器环境中，采用浏览器/小程序客户端访问服务端的方式运行。学生通过微信小程序或移动端 Webview 访问系统，管理员通过 PC 浏览器访问后台管理系统。服务端通过 Nginx 或 API Gateway 提供统一入口，并将请求转发至后端应用服务。

后端应用服务负责调用 AI 问答服务、文件解析服务、PDF 生成服务和日志审计服务，并统一访问 Kingbase 数据库和文件存储。数据库服务器用于保存结构化业务数据，文件存储用于保存政策文件、模板、成绩单和生成后的证明 PDF。系统还可以接入微信平台 and 邮件服务，用于发送提醒和通知。

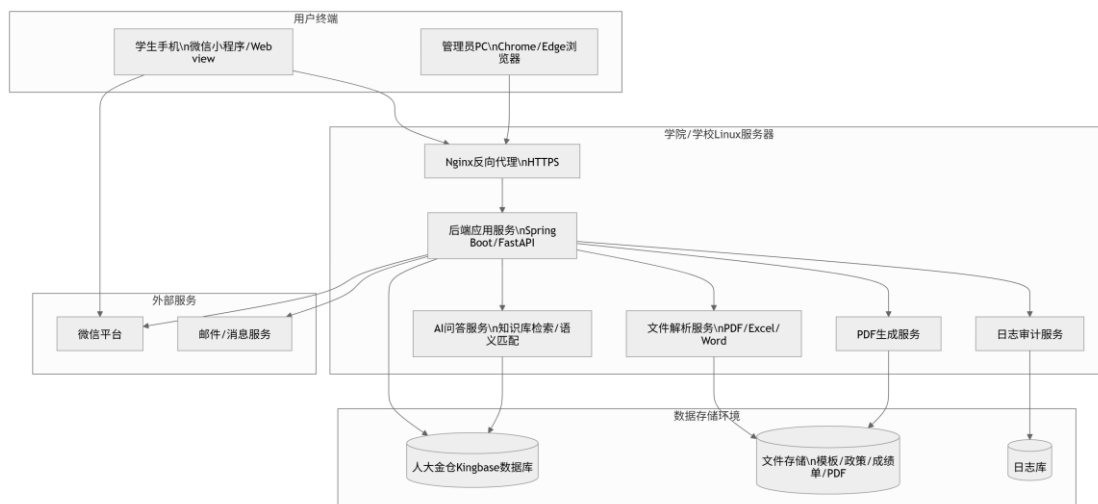


图 3-23 系统部署图

图 3-23 展示了系统的目标部署结构。客户端不直接访问数据库和文件存储，所有请求均通过服务端进行权限校验和业务处理。Nginx/API Gateway 负责统一入口、HTTPS 访问和请求转发，后端服务负责具体业务逻辑，Kingbase 数据库和文件存储负责数据持久化。

在实际部署时，应将数据库账号、文件存储路径、AI 服务地址、邮件服务配置等参数写入配置文件或环境变量，避免硬编码在程序中。系统应保留必要的运行日志和错误日志，便于后续排查问题。对于审批、导入导出和敏感信息查看等关键业务，还需要保留业务审计日志，以满足系统安全性和可追溯性要求。

通过上述部署设计，系统能够支持学生端和管理端的多端访问，并在保证权限隔离、数据安全和模块独立性的基础上，为后续功能扩展提供稳定运行环境。